Aula 14 - Relacionamentos

Programação Orientada a Objetos
Prof. Matheus Avila Moreira de Paula
matheusavila@cefetmg.br

Engenharia de Computação
Centro Federal de Educação Tecnológica Leopoldina

Agregação

- Definição:
  - Uma forma especial de associação entre o todo e suas partes, no qual o todo é composto de partes. Associação na qual um objeto “agregado” é feito de componentes - os componentes fazem parte do agregado
  - Relaciona objetos diferentes que compõem um agregado
    - Os componentes podem ser do mesmo tipo ou de tipos diferentes
- Há dois tipos de Agregação:
  - Agregação Simples
  - Composição
    - É uma agregação mais forte

Agregação Simples

Agregação Simples

- Definição
  - Relacionamento entre DUAS classes que estabelece que uma instância de uma agrupa uma ou mais instâncias da outra

- Caracterizada por:
  - Dependência fraca entre todo e parte
  - Se todo morrer, as partes podem continuar a existir
  - Indica qual é o todo e quais as partes
  - Não vincula o tempo de vida do todo a suas partes
  - Uma parte pode ser compartilhada por vários todos
    - Ex.: uma Parede pode ser parte de um ou vários Cômodos

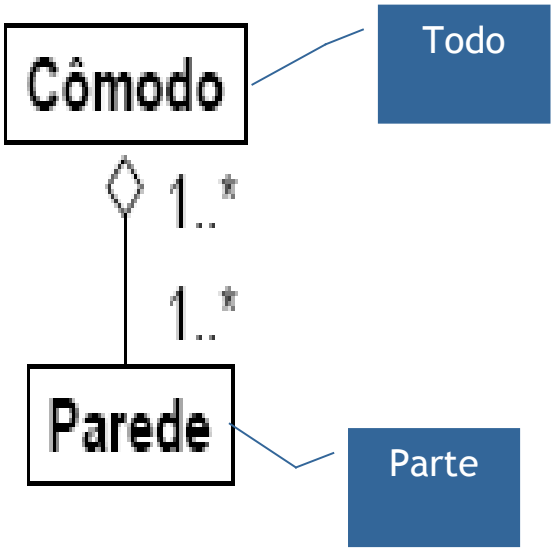
Agregação Simples

- Observação
  - Espécie vaga de associação.
  - Não possui semântica significativa na UML (comparada a uma associação)
  - Recomenda-se não se preocupar com seu uso
    - Use “composição” quando necessário expressar relacionamento **todo/parte**

Agregação Simples

Se o “Cômodo” deixar de existir, as paredes (uma ou mais) podem continuar a existir

Uma única parede não configura um cômodo, mas continua a ser parede (pode existir)



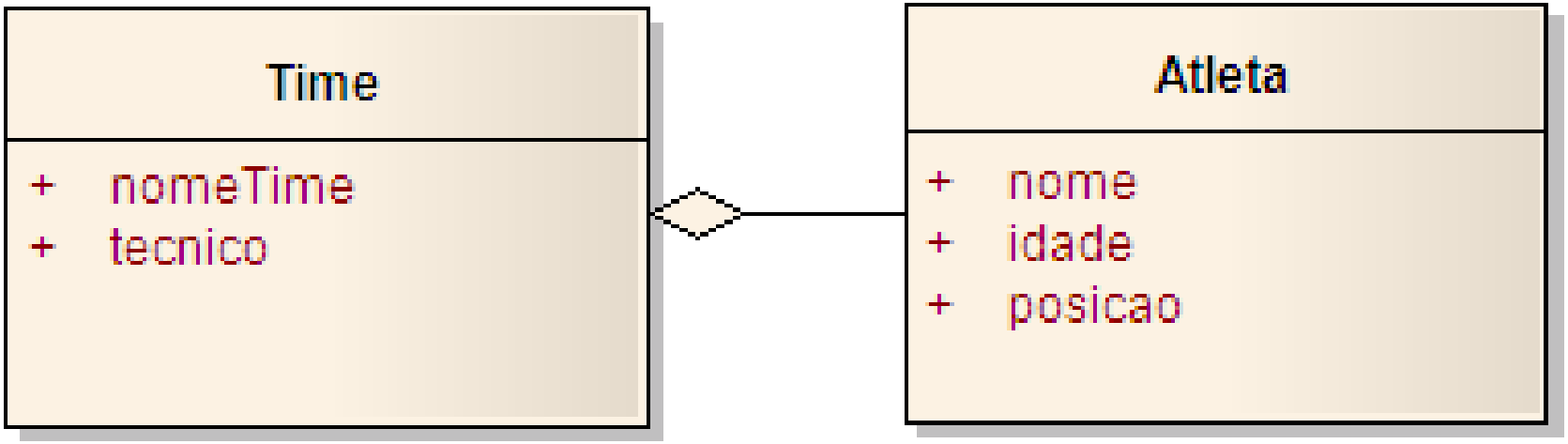
Agregação Simples

Um livro é composto por uma ou mais páginas

Um acervo é composto de um ou mais livros

Agregação Simples

- Um time **é formado por** atletas, ou seja, os atletas **são parte** integrante de um time
- mas os atletas existem independentemente de um time existir.



Agregação

Outras características:

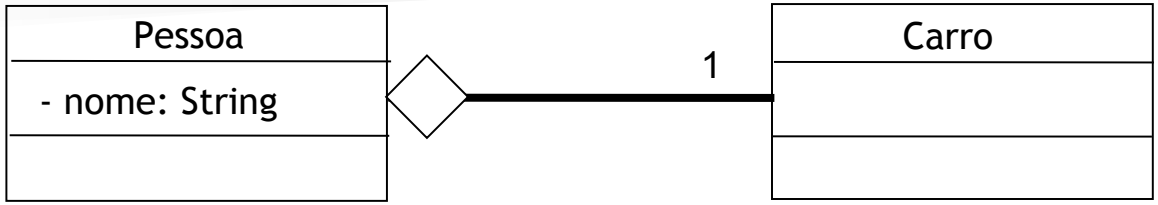
- Os objetos partes podem existir sem serem parte do todo
  - Ex: Carro e rodas
  - roda pode ser retirada, antes de se destruir o carro
  - Roda retirada pode ser colocada em outro carro

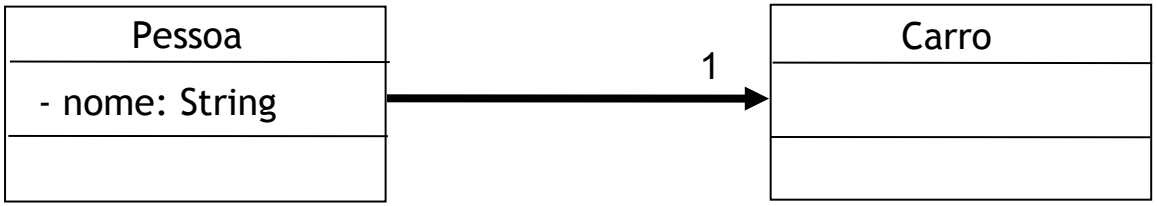
Agregação

Outras características:

- São associações anti-simétricas:
  - Objeto não pode ser direta ou indiretamente parte de si mesmo.

Agregação






```
3  public class Pessoa {
4
5      private String nomePessoa;
6      private Carro carro;
7
8      public Pessoa(String nome, Carro carro){
9          this.nomePessoa = nome;
10         this.carro = carro;
11     }
12 }
```

Agregação

```
3  public class Pessoa {
4
5      private String nomePessoa;
6      private Carro carro;
7
8      public Pessoa(String nome, Carro carro){
9          this.nomePessoa = nome;
10         this.carro = carro;
11     }
12 }
```

Observe que a classe Pessoa está recebendo um objeto do tipo Carro como parâmetro. Logo, este objeto foi criado/instanciado em um momento qualquer, diferente de Pessoa. Uma vez criado, ele foi passado para pessoa.
Observe, ainda, que se a pessoa "morrer" (for destruída no contexto do sistema), o carro continuará existindo – ou seja, ele foi instanciado e a sua instância ainda existirá, idependente do que acontecer com pessoa. Haverá, apenas, uma quebra da ligação entre pessoa e carro.

Agregação

Observação:

Em termos de **implementação** não existe diferença entre:

agregação simples e associação

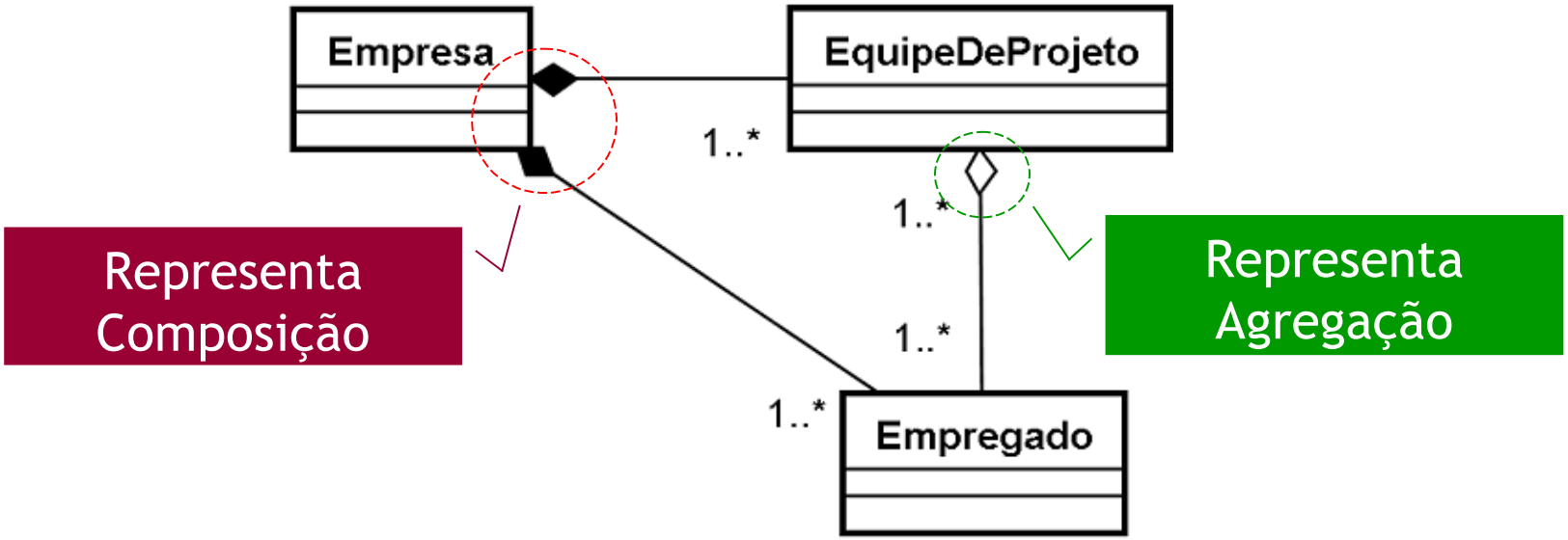
Composição

Composição

- Definição
  - Um tipo de relação de agregação com restrições na ligação entre parte e agregado

- Caracterizada por:
  - Uma instância da parte é agregada por uma única instância do agregado
  - Instanciação do agregado precede a instanciação da parte
  - Dependência existencial (dependência forte)
    - Todo deixa de existir = partes também deixam de existir
  - As partes não podem ser compartilhadas
  - O todo é responsável pelas suas partes

Composição


Empresa
EquipeDeProjeto
1..*
1..*
1..*
1..*
Empregado
Representa Composição
Representa Agregação

Composição

- Dependência existencial
  - Criação:
    - Cardinalidade 1: as partes e a composição são criadas ao mesmo tempo
    - As partes sem cardinalidade fixa podem ser criadas após a composição
  - Destruição:
    - As partes morrem ao se destruir a composição (morrem se o todo morre)
    - As partes podem ser removidas antes da morte do objeto composto

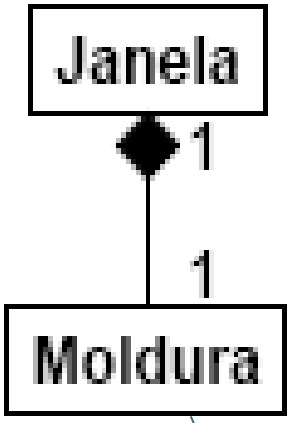
Composição

- As partes não podem ser compartilhadas
  - Um objeto poderá ser parte de somente uma composição em um determinado momento.
    - Ex.: uma Moldura pertence a uma única Janela

- O todo é responsável pelas suas partes
  - O objeto composto deve gerenciar a criação e a destruição de suas partes
  - Ex:
    - Ao criar uma Moldura em um sistema de janelas, a moldura deve ser anexada à janela que a conterá. Quando destrói a janela, o objeto Janela, por sua vez, deve destruir sua parte Moldura

Composição

Seria possível criar uma abstração para um outro sistema, admitindo que uma moldura pode existir sem uma janela. Neste caso, tem-se uma agregação (◊)

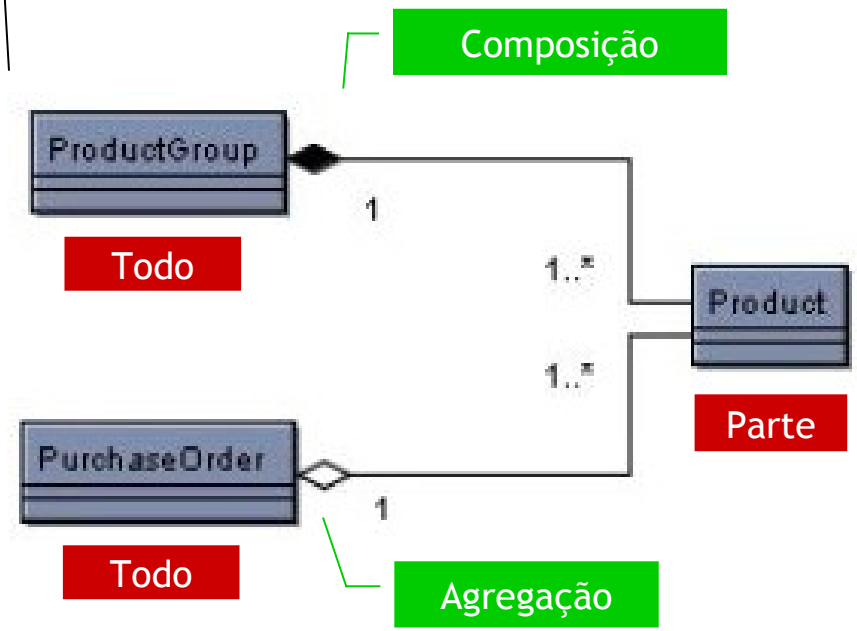


No sistema de Janelas, uma moldura só existe se existir um objeto Janela - toda moldura precisa de uma janela para existir

Agregação e Composição

Um ProductGroup (grupo de produtos) é composto por um ou vários produtos. Se o grupo “morre”, então os produtos também “morrem”. Se o grupo for visto como um objeto caixa de produtos e se essa for destruída, os produtos nela também o serão

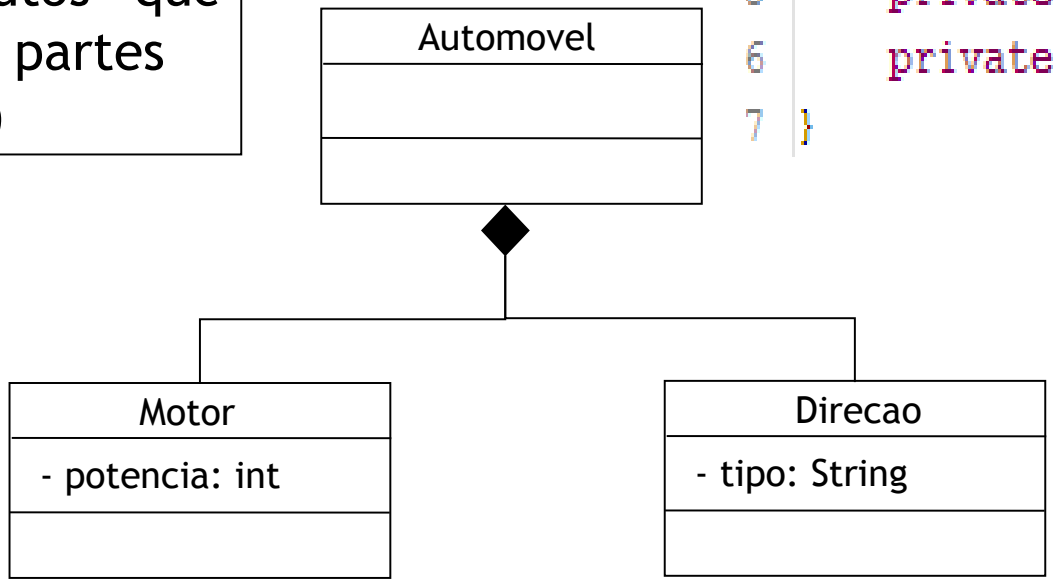
Uma ordem de compra é composta de vários produtos. Se a ordem é destruída (cancelada, por exemplo), os produtos ainda continuaram “vivos”para serem comprados depois



Implementação Composição

Composição

Existe a necessidade de se criar atributos que identifiquem as partes (na classe todo)




```
3 public class Automovel {
4
5     private Motor motor;
6     private Direcao direcao;
7 }
```

```
3 public class Motor {
4
5     private int potencia;
6 }
```

```
3 public class Direcao {
4
5     private String tipo;
6 }
```

Composição

```
3  public class Motor {
4
5      private int potencia;
6
7      public int getPotencia() {
8          return potencia;
9      }
10
11     public void setPotencia(int potencia) {
12         this.potencia = potencia;
13     }
14 }
```

```
3  public class Direcao {
4
5      private String tipo;
6
7      public String getTipo() {
8          return tipo;
9      }
10
11     public void setTipo(String tipo) {
12         this.tipo = tipo;
13     }
14 }
```

Métodos set e get para os atributos das classes que formam as partes do todo

Composição

```
3  public class Automovel {
4
5      private Motor motor;
6      private Direcao direcao;
7
8      public Motor getMotor() {
9          return motor;
10     }
11     public void setMotor(Motor motor) {
12         this.motor = motor;
13     }
14     public Direcao getDirecao() {
15         return direcao;
16     }
17     public void setDirecao(Direcao direcao) {
18         this.direcao = direcao;
19     }
20 }
```

Até o momento, nada diferente em relação às associações vistas em momentos anteriores. Uma classe está associada a outras, possui atributos que as representam e métodos set e get (para alterá-los e recuperá-los)

Composição

Este método construtor da classe Automóvel chama o construtor de Motor sem parâmetro.

Este método construtor da classe Automóvel chama o construtor de Motor com parâmetro.

```
class Automovel {
      private Motor motor;

public Automovel(){
      motor = new Motor();
    }

public Automovel(int potencia){
      motor = new Motor(potencia);

    }
}
```

É no construtor que representamos a idéia de composição mais claramente

Composição

```
3  public class Principal {
4
5      public static void main(String args[]){
6
7          Automovel a1 = new Automovel(500);
8          Automovel a2 = new Automovel();
9
10         System.out.println(a1.getMotor().getPotencia());
11         System.out.println(a2.getMotor().getPotencia());
12     }
13 }
```

Observe que este construtor recebe um parâmetro que inicializa um outro construtor – ou seja, Automóvel recebe 500, que será utilizado no construtor de Motor. Já discutimos situações como esta, onde um construtor recebe argumentos que serão repassados para outro construtor (aula de implementação de associações).
O outro construtor não é diferente do que já foi visto

Composição

```
3  public class Automovel {
4
5      private Motor motor;
6      private Direcao direcao;
7
8      public Automovel(){
9          motor = new Motor();
10     }
11
12     public Automovel(int potencia){
13         motor = new Motor(potencia);
14     }
```

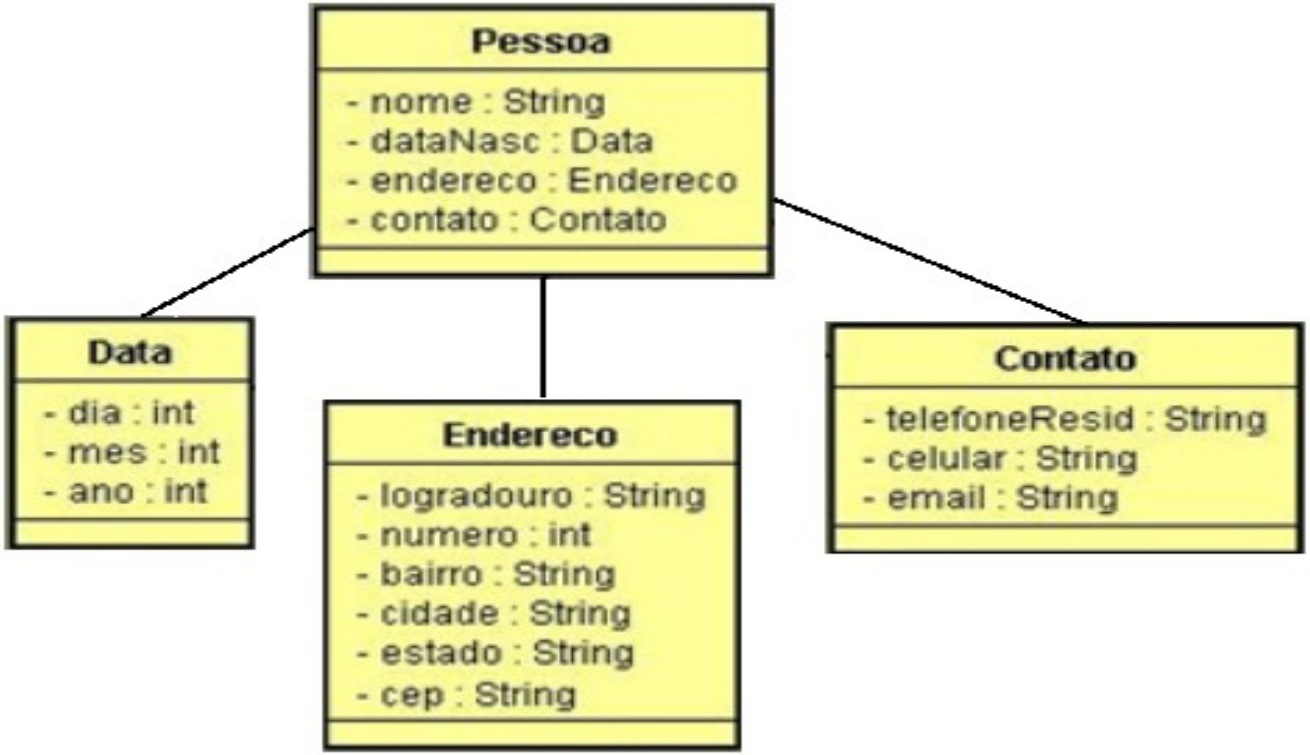
Composição

```
3  public class Motor {
4
5      private int potencia;
6
7      public Motor(){
8          this.setPotencia(1000);
9      }
10
11     public Motor(int potencia){
12         this.setPotencia(potencia);
13     }
14
15     public int getPotencia() {
16         return potencia;
17     }
18
19     private void setPotencia(int potencia) {
20         this.potencia = potencia;
21     }
22 }
```

Prática

Prática composição

Crie as classes apresentadas no diagrama abaixo e aplique a Composição para a classe Pessoa, que além de possuir um atributo Nome será composta pelas classes Data, Endereço e Contato para os atributos dataNasc, endereco e contato, respectivamente.

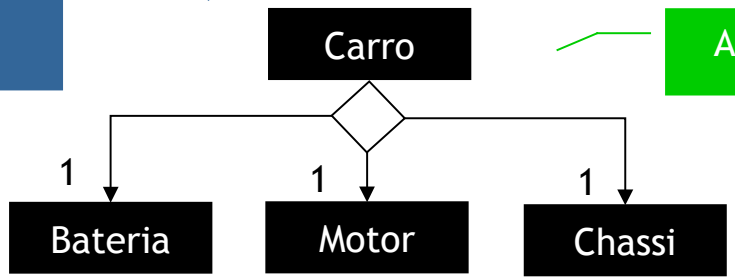


Tipos – Agregação e Composição

Agregação e Composição - Fixa

- Agregação/Composição Fixa
  - Tem estrutura fixa: a quantidade de tipos e de subpartes é predefinida
  - Ex: Agregação
    - Um carro é composto por motor, rodas, portas

Se o todo (carro) é destruídos, suas partes podem ser aproveitadas - podem ser retiradas antes e colocadas em outro carro

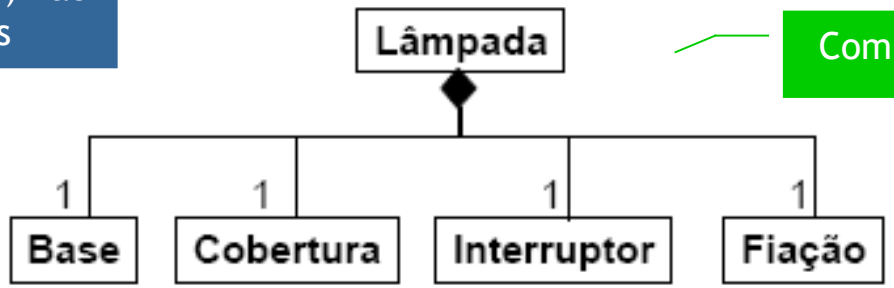


Agregação Fixa

Agregação e Composição - Fixa

- Composição Fixa
  - Ex: Uma Lâmpada tem uma base, uma cobertura, um interruptor e fiação

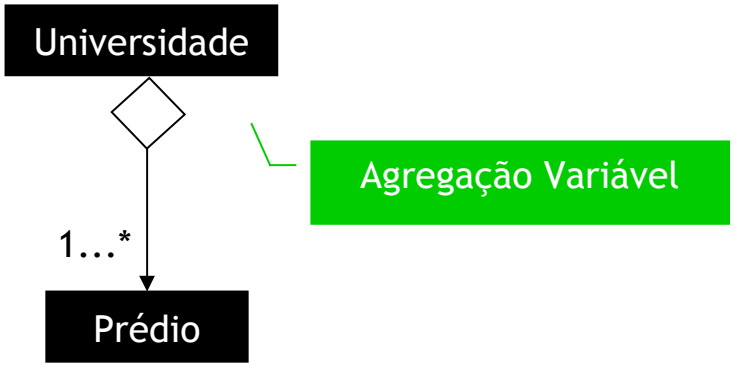
Em um dado sistema, se o Objeto lâmpada for destruído, suas partes (objetos partes) não podem ser reutilizadas



Composição Fixa

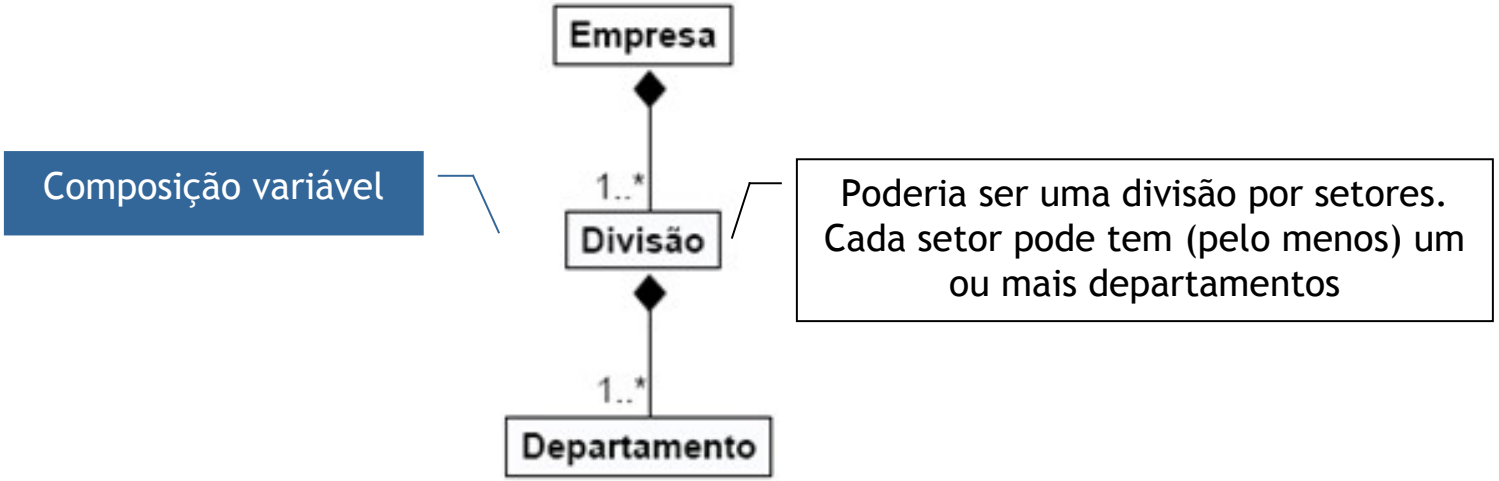
Agregação e Composição - Variável

- Agregação/Composição Variável
  - O número de níveis é finito. Contudo, o número de partes pode variar
  - Exemplo Agregação



Agregação e Composição - Variável

- Composição Variável
  - Uma empresa pode ter muitas divisões
  - Cada divisão pode ter múltiplos departamentos.



Agregação e Composição - Recursiva

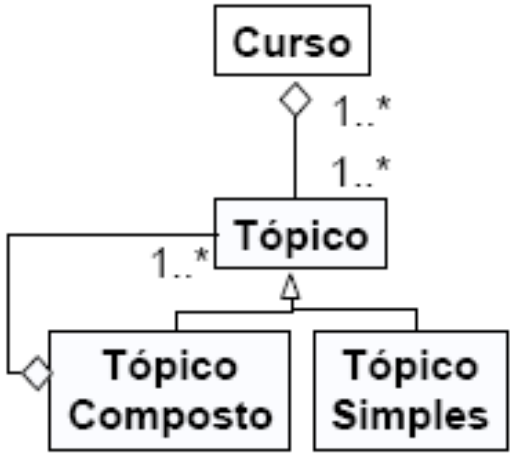
- Agregação/Composição Recursiva
  - Contém (direta ou indiretamente) uma instância do mesmo tipo do agregado
  - A quantidade de níveis é ilimitada

  - Exemplo Agregação
    - Um curso é uma agregação de tópicos. Os tópicos podem ser compostos por subtópicos, terminando em tópicos simples. Os tópicos podem ser aninhados até uma profundidade arbitrária.

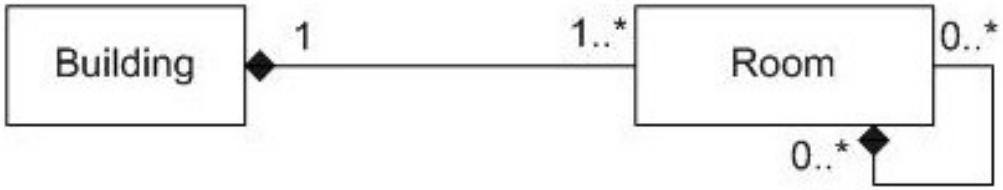
Agregação e Composição - Recursiva

É possível dividir um Tópico Composto em outros Tópicos, que podem ser "Tópicos Compostos"ou "Simples"

Agregação Recursiva



Agregação e Composição - Recursiva



Composição recursiva

Uma construção (prédio) é composto de (pelo menos) uma ou várias dependências (salas). Uma sala, por sua vez, pode ser composta de nenhuma ou várias outras salas.

Basta imaginar o atual DCC (tem-se uma sala única com várias salas de professores)

Composição

- Chamadas a métodos contrutores
  - Os métodos construtores das classes componentes (Motor,Direção) que fazem parte da classe composta (Automóvel), podem ser chamados de três maneiras diferentes:
    - Caso 1: chamadas nos construtores da classe que é composta;
    - Caso 2: chamadas em qualquer método da classe que é composta;
    - Caso 3: chamadas fora da classe que é composta.

Composição

Os construtores da classe motor são chamados dentro dos construtores da classe Automóvel.

```
3  public class Automovel {
4
5      private Motor motor;
6      private Direcao direcao;
7
8      public Automovel(){
9          motor = new Motor();
10     }
11
12     public Automovel(int potencia){
13         motor = new Motor(potencia);
14     }
```

Composição

```
3  public class Automovel {
4
5      private Motor motor;
6      private Direcao direcao;
7
8      public Automovel(){
9          motor = new Motor();
10     }
11
12     public Automovel(int potencia){
13         motor = new Motor(potencia);
14     }
15
16     public void ligarAutomovel(){
17         motor = new Motor();
18     }
19
20     public void ligarAutomovel(int potencia){
21         motor = new Motor(potencia);
22     }
```

Os construtores são chamados em qualquer método da classe que é composta

Composição

```
3  public class Principal {
4
5      public static void main(String args[]){
6
7          Automovel auto = new Automovel();
8          Motor motor = new Motor();
9
10         auto.setMotor(motor);
```

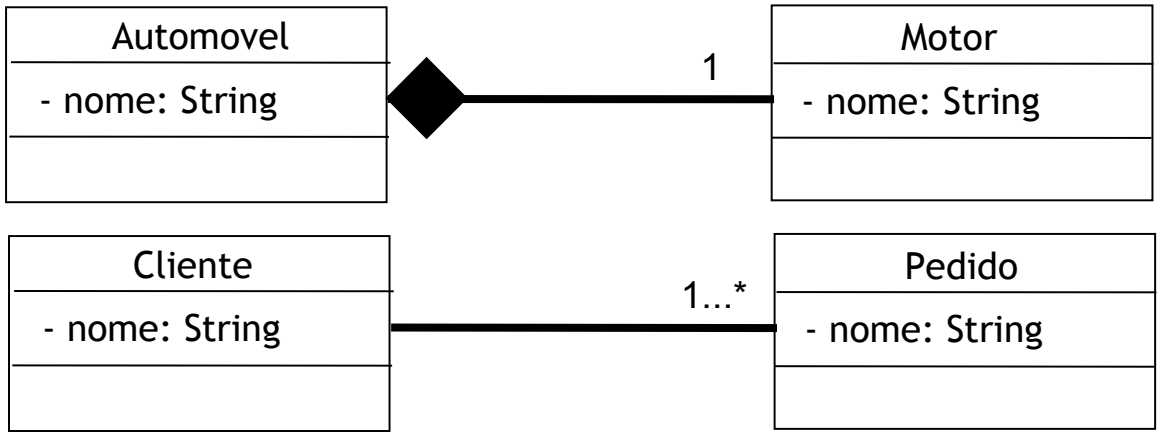
Observe que o construtor da classe Motor foi chamado de fora da classe Automóvel, dentro do método main()da classe. Ou seja, antes de usar o métodosetMotor() o objeto motor não tem nenhum vínculo com o objeto automóvel.

Associação e Composição

```
3 public class Automovel {
4
5     private Motor motor;
6     private Direcao direcao;
7
```

```
3 public class Cliente {
4
5     private Pedido pedido;
6
7 }
```

Associação e Composição



Associação e Composição

```
3  public class Automovel {
4
5      private Motor motor;
6      private Direcao direcao;
7
8      public Automovel(){
9          motor = new Motor();
10     }
11
12     public Automovel(int potencia){
13         motor = new Motor(potencia);
14     }
```

Composição

Associação

```
3  public class Cliente {
4
5      private Pedido pedido;
6
7      public Cliente(Pedido pedido){
8          this.pedido = pedido;
9      }
```

Associação e Composição

```
3  public class Automovel {
4
5      private Motor motor;
6      private Direcao direcao;
7
8      public Automovel(){
9          motor = new Motor();
10     }
11
12     public Automovel(int potencia){
13         motor = new Motor(potencia);
14     }
```

Composição

Associação

```
3  public class Empregado {
4
5      private Projeto projeto;
6
7      public Empregado(){
8          projeto = new Projeto();
9      }
10 }
```

Observe que, para este caso, não há diferenças de implementação entre composição e associação (ilustrada na igura por 1→1)

Agregação e Composição

Agregação e Composição

```java
3  public class Pessoa {
4
5      private String nomePessoa;
6      private Carro carro;
7
8      public Pessoa(String nome, Carro carro){
9          this.nomePessoa = nome;
10         this.carro = carro;
11     }
12 }
```

```java
3  public class Automovel {
4
5      private Motor motor;
6      private Direcao direcao;
7
8      public Automovel(){
9          motor = new Motor();
10     }
11
12     public Automovel(int potencia){
13         motor = new Motor(potencia);
14     }
```

Agregação e Composição

- Agregação
  - Um objeto B existe fora de outro objeto A. B é criado e, então, passado como um argumento para o contrutor de A.
    - Ex: Pessoa e Carro. O carro é criado em um contexto diferente e torna-se uma propriedade de pessoa
- Composição
  - Um objeto somente existe, ou somente faz sentido, dentro de outra, como uma parte. Ex: Pessoa e Cérebro. Não se cria um cérebro para, então, passá-lo para uma pessoa.

Resumo

Resumo Relacionamentos

- Dependência (seta tracejada):
  - É o relacionamento mais fraco da UML, simplesmente diz que um elemento client (que atira a seta) é semanticamente ou estruturalmente dependente do supplier (que recebe a seta). Exemplo: Produto e AliquotaImposto.

- Associação (linha contínua):
  - As duas classes são independentes e podem trabalhar juntas. A visibilidade destaca qual lado consegue enxergar as extremidades da associação, mas ambas as classes podem estabelecer o relacionamento. Exemplo: Cliente e Fornecedor.

Resumo Relacionamentos

- Agregação Simples (linha contínua com diamante branco):
  - A classe que possui o diamante branco controla a associação. Exemplo: Equipe e Pessoa. A equipe é quem pode estabelecer o relacionamento e pessoas são adicionadas à equipe. Uma pessoa por sí só não pode "entrar na equipe". É a equipe que manda no relacionamento. Contudo, as classes são independentes. Pessoas existem fora de equipes.

Resumo Relacionamentos

- Composição (linha contínua com diamante negro):
  - A composição é uma agregação mais forte. A classe que possui o diamante controla a associação, e além disso, a outra classe só pode existir associada à classe que tem o diamante e não pode estar associada a outras instâncias. Exemplo: Nota e ItemNota, Pedido e ItemPedido.

Referências

- Material da professora Bárbara de Melo Quintela (UFJF)
- BARNES, David J.; KÖLLING, Michael. Programação orientada a objetos com Java: uma introdução prática usando o BlueJ. 4 ed. São Paulo: Pearson Prentice Hall, c2009. ISBN: 9788576051879